A Distributed Edge–Cloud System for Bloodstain Pattern Analysis

Emanuele Ugolotti
Course: IoT and Mobile Systems
[Unimore]

September 18, 2026

Abstract

The system is composed of two ESP32-S3 microcontrollers acting as an *Acquisition Node* and a *Gateway*, an Android mobile application, and a containerized backend. This report focuses primarily on the IoT and mobile layers—specifically the embedded firmware, communication protocol, and Android app—covering the cloud backend only where it interacts with the edge devices.. We discuss the main design decisions, the engineering challenges encountered during development (particularly around radio coexistence and memory constraints on the microcontrollers), and the results of functional and reliability testing.

1 Introduction

1.1 Motivation and Scope

Bloodstain Pattern Analysis (BPA) is a forensic technique used to reconstruct the events of a crime from the shape, size, and distribution of bloodstains. This project implements such a pipeline as a distributed edge–cloud system, built for a university course covering IoT and mobile systems.

The system integrates three parts: an *IoT layer* (two ESP32-S3 boards communicating over a custom radio protocol), a *mobile layer* (an Android application capable of both receiving data and running inference locally), and a *cloud layer* (a containerized backend providing storage, more powerful inference, and adversarial-robustness testing). This report emphasizes the first two layers, since they are the subject of the course, while treating the cloud backend as supporting infrastructure whose interface is described where relevant.

2 System Overview

Figure 1 shows the overall architecture. An *Acquisition Node* (an ESP32-S3-EYE board with a camera and a small display) either selects a reference image from a shared dataset or captures a real photograph, and sends it over ESP-NOW — a connectionless, low-overhead radio protocol — to a *Gateway* (a second ESP32-S3 board). The Gateway is also connected to a WiFi network, and forwards each acquisition over HTTP to either the cloud backend or, as a fallback, directly to the mobile application, which exposes an embedded HTTP server for this purpose. The mobile application can also query the cloud backend directly to retrieve previously computed results.

Two acquisition modes are supported end-to-end: *reference* mode, in which the Acquisition Node sends only the filename of an image already present in a shared dataset, and *captured* mode, in which a real photograph taken by the on-board camera is transmitted in full.

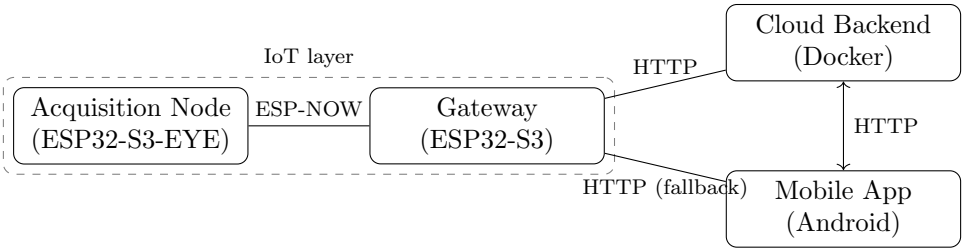


Figure 1: High-level system architecture. The IoT layer (dashed box) communicates with the mobile application either directly or through the cloud backend.

3 Tools and Technologies

Table 1 summarizes the technologies used in each layer of the system.

Layer	**Technologies**
IoT firmware	ESP-IDF (v5.5), C, FreeRTOS, ESP32-S3 (two boards: one with an OV2640 camera and LCD, one plain), ESP-NOW radio protocol
Mobile app	Android SDK, Java, TensorFlow Lite, NanoHTTPD (embedded HTTP server), SQLite
Cloud backend	Python, FastAPI, PostgreSQL, Docker / Docker Compose, Docker Swarm (deployment demonstration), TensorFlow/Keras (VGG16), custom implementations of the Boundary Attack and SimBA adversarial attacks

Table 1: Technologies used per system layer.

4 IoT Subsystem

The IoT subsystem is the core contribution of this course project: two physical microcontrollers that communicate over a radio link with no shared network infrastructure, implementing reliability mechanisms (checksums, retries, timeouts) entirely in application code.

4.1 Acquisition Node

The Acquisition Node runs on an ESP32-S3-EYE board, which integrates an OV2640 camera and a small LCD. On boot, it connects to WiFi (required only to enable the underlying radio for ESP-NOW, not for internet access) and registers the Gateway as an ESP-NOW peer. In automatic mode it cycles through a small shared dataset, sending one *reference* packet every few seconds; a physical button (mapped to a plain GPIO pin rather than the board's analog-multiplexed buttons, for reasons discussed in Section 4.4) can trigger a real photo capture, or toggle automatic mode on and off with a long press.

A captured photograph is first stored as a raw RGB565 frame (required for the live preview), then converted to a compressed JPEG only at the moment of capture, since a raw frame is roughly ten times larger than its compressed equivalent. The resulting JPEG (typically 7–12 KB) is far larger than the ∼250-byte payload limit of a single ESP-NOW packet, so it must be fragmented.

4.2 Fragmentation Protocol

Since ESP-NOW provides no built-in fragmentation (unlike TCP/IP, which handles arbitrarily large payloads transparently), a small protocol was designed on top of it. Each transfer consists of one *header* packet, containing the total image size, the number of fragments, and a CRC16 checksum of the whole image, followed by a sequence of *data* packets, each carrying a 200-byte slice of the JPEG and its own index. Every packet (in both this protocol and the simpler *reference*-mode packet) carries its own CRC16 to detect in-transit corruption. Figure 2 shows the message sequence for a successful captured-photo transfer.

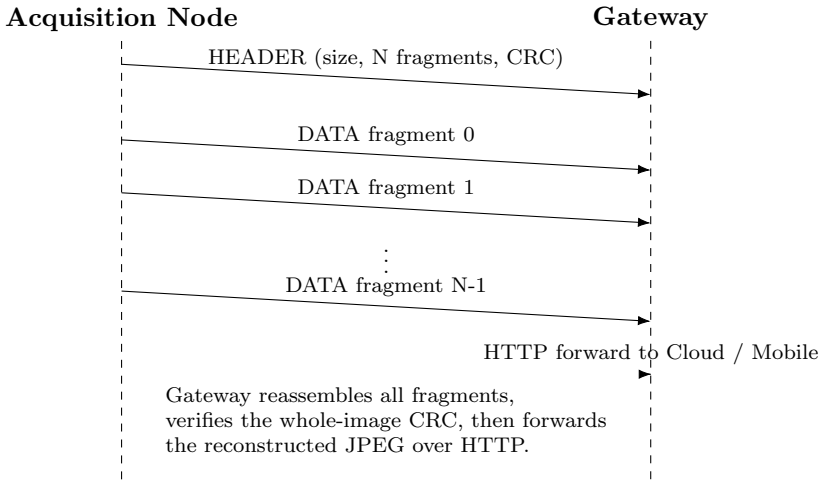


Figure 2: Fragmentation and reassembly sequence for a captured photograph.

4.3 Gateway

The Gateway receives both packet types over ESP-NOW. For *reference* packets, it simply forwards a small JSON payload. For fragmented photographs, it maintains a single in-progress reassembly buffer (allocated once the header arrives), copies each incoming fragment to its correct offset, and — once all fragments have arrived — recomputes the CRC over the reassembled buffer and compares it to the value declared in the header before forwarding. If the CRC does not match, or if fragments stop arriving for more than a fixed timeout (monitored by a dedicated background task), the attempt is declared failed and reported to the backend rather than silently dropped.

Offloading logic: the Gateway prefers the cloud backend when a quick health check succeeds, and falls back to sending the acquisition directly to the mobile application otherwise. Reference packets that fail to reach either destination are placed in a small retry queue and re-attempted every 30 seconds; captured photographs, being too large to buffer indefinitely in this queue, use a smaller number of immediate retries with short delays instead.

4.4 Engineering Challenges

Development revealed several hardware and concurrency issues, highlighting key insights into the IoT implementation:

- **ADC/camera hardware conflict.** The board's four auxiliary buttons share an analog-to-digital converter channel. Creating them through the vendor's board-support package started an

internal polling timer that conflicted with the camera driver's use of the same hardware, causing a crash. The fix was to use the board's dedicated `BOOT` button instead, which is wired to a plain digital GPIO pin and shares no resource with the camera.

- **Buffer over-read in the live preview.** The preview routine copied a fixed number of bytes from each captured camera frame into the display buffer, based on the display's resolution rather than the camera's actual frame size; when the two did not match, this silently read past the end of the camera's own buffer on every frame, intermittently corrupting nearby heap memory. It was fixed by copying only the number of bytes actually available in each frame.
- **Blocking network calls inside a radio callback.** On the Gateway, HTTP forwarding was originally invoked directly from the ESP-NOW receive callback. Because a forwarding attempt (including retries) could take up to 30–40 seconds, this blocked the underlying network stack for the same duration, causing seemingly unrelated connection failures. The fix moved all forwarding work to a dedicated FreeRTOS task fed by a queue, so the radio callback always returns immediately. A shared mutex was also introduced to protect the reassembly state, which was previously accessed without synchronization from both the radio callback and an independent timeout-checking task.

5 Mobile Application

The Android application (Java) serves three roles: it can receive acquisitions directly from the Gateway, classify images entirely on-device, and read the result from the cloud backend client.

5.1 Architecture

The app is organized around a bottom-navigation structure with five main screens: Home (server status and most recent result), History (a local SQLite log of every classification, from any source), Manual Test (classify a gallery image or a new photo), Adversarial Test (run an adversarial attack against a stored image), and Settings.

5.2 On-device Inference

The application embeds a TensorFlow Lite conversion of the same VGG16 model used by the backend, with identical preprocessing (RGB to BGR channel order and ImageNet mean subtraction, matching the model's training pipeline). An embedded HTTP server (NanoHTTPD) exposes an ingest endpoint mirroring the cloud backend's schema, so the Gateway can treat the phone as a fallback compute target: when the cloud is unreachable, the exact same acquisition message is instead sent to, and classified by, the phone.

5.3 Cloud Synchronization and Live View

The app synchronizes cloud results into its local history and features a read-only 'live view' to query the backend on demand. This allows users to view classifications and failed attempts without local storage, serving as a prototype for a mobile remote control.

5.4 Adversarial Testing Interface

A dedicated screen lets the user select an image (from the gallery or from local history) and request either a Boundary Attack or a SimBA attack against it, run asynchronously as a background service

so the attack (which can take several minutes) does not block the UI; a notification is shown on completion, and the outcome (whether the attack changed the predicted class), together with the probability distributions before and after the attack, is logged to a dedicated attack history.

6 Cloud Backend

The cloud backend is a small set of Docker containers: a FastAPI service exposing the ingest and query endpoints described above, a PostgreSQL database, and a service that hosts the VGG16 model both for plain inference and for the two adversarial attacks. The ingest endpoint is reused by both acquisition modes and never fails the caller's request because classification failed — inference is attempted after the acquisition is already safely stored, so a temporarily unreachable model service degrades gracefully rather than causing data loss. A minimal, framework-free web dashboard was also added to visualize stored results, including thumbnails and full per-class probability breakdowns, directly from a browser. As a deployment exercise, an equivalent Docker Swarm stack definition was also produced and tested alongside the Compose-based one used for development.

7 Evaluation

7.1 Functional Testing

The full pipeline was exercised end-to-end on physical hardware multiple times: image acquisition (both reference and captured modes), fragmentation, radio transmission, reassembly with checksum verification, HTTP forwarding to both the cloud backend and the phone, server-side inference, and finally retrieval of the result through the mobile application's history view and the web dashboard. All stages were confirmed to work correctly after the fixes described in Section 4.4.

7.2 Performance Observations

Cloud-side inference on a single image (through the shared model service) completed in under one second in all observed runs, correctly classifying a representative test image with 99.97% confidence. A full captured photograph (roughly 8 KB after JPEG compression) was reliably transmitted as 38–48 fragments; occasional individual-fragment loss over the radio link was observed, at a rate low enough that the retry and timeout mechanisms described above were sufficient to keep failures visible and non-silent rather than to eliminate them outright.

7.3 Known Limitations

Given the scope of the course project, we accepted two main limitations:

- The Gateway's retry queue exists only in RAM and is lost on reboot; persisting it would require flash storage, which was judged unnecessary for a prototype of this scale.
- A rare heap-corruption crash inside the third-party JPEG-encoding library used on the Acquisition Node was reduced substantially (through the fixes in Section 4.4 and a short mandatory delay between consecutive captures) but not eliminated with full certainty; it is documented as a known, low-frequency issue.

8 Conclusion and Future Work

The IoT contribution in particular required designing a custom fragmentation protocol over a connectionless radio link and resolving several low-level hardware and concurrency issues that are not visible from a purely software perspective. Natural extensions include persisting the Gateway's retry state across reboots, replacing HTTP with a publish/subscribe protocol such as MQTT between the Gateway and the backend, and extending the cloud dashboard to trigger adversarial attacks remotely on already-stored images without requiring the phone to download and re-upload them.